

# 深圳大学实验报告

课程名称: 基于 Web 的编程

实验项目名称: JS 综合实验

学院: 计算机与软件学院

专业: 软件工程（腾班）

指导教师: BASKER GEORGE

报告人: 洪子敬 学号: 2022155033 班级: 腾班

实验时间: 2025 年 05 月 06 日至 06 月 24 日

实验报告提交时间: 2025 年 6 月 11 日 星期三

教务处制

## 实验目的与要求:

**目的:** 能够使用 JS, 构思并设计、实现较完整的用户注册、登录等功能模块。

**要求:** 1、设计实现自己网站的用户注册和登录模块。

2、要求注册和登录模块用户体验良好, 做好安全防护方面的设计。

## 方法、步骤:

要完成本实验, 依据实验要求进行分解, 需要完成的实验步骤是:

1. 完成上机实验, 包括
  - 1.1 数字运算和循环
  - 1.2 偶数平方和
  - 1.3 动态计数器
  - 1.4 商品清单生成器
  - 1.5 学习主页优化
  - 1.6 星级评分
  - 1.7 待办事项
  - 1.8 学习主页再次优化
  - 1.9 英雄快跑游戏
  - 1.10 拖拽排序
  - 1.11 通讯录管理
  - 1.12 WebWorker
  - 1.13 用户注册
2. 设计并实现某主题网站的账户模块, 包括
  - 2.1 构思、设想
  - 2.2 设计实现及安全优化
  - 2.3 效果呈现
  - 2.4 思考小结

特别注意:

1. 不要贴大篇幅的源码, 连续代码行不超过 20 行。
2. 不要贴大体积的图片, 最后提交的文档不能超过 20M。
3. 作业提交: 提交 doc 或 pdf 文件任一即可, 无需提交源代码;
4. 特别注意: 提交文件不要打包, 不要超过 20M (**高分辨率截图容易导致一个文件几百 M, 特别注意不要出现这种情况**)

## 实验过程及内容:

1. 上机实验完成情况:

- 1.1 数字运算和循环

- 1.1.1 实验过程 (包括是否遇到问题及如何解决)

解答: 这个实验比较简单, 只需要根据流程编写相应的代码即可, 而 JavaScript 的基础语法与 C++ 比较接近, 整体来说还是比较好些的; 只是需要注意数组的定义的同样是通过 New Array() 得到的, 并通过 push 方法进行元素的添加; 同时控制台的输出通过 console.log() 进行打印即可, 语法和 C 语言的 printf 语法类似。

- 1.1.2 实验结果

解答：运行补充完的脚本，可以得到如图 1 所示的结果，可以看到数组中填充了  $1^2, 2^2, \dots, 10^2$  的元素，且求和结果为 385，与预期一致，证明代码编写成功。

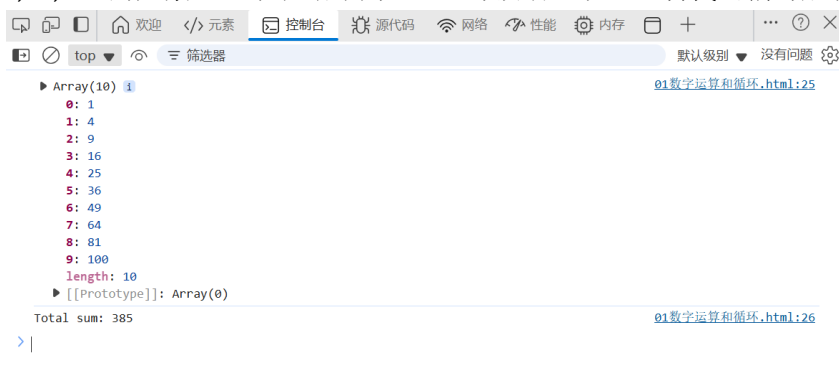


图 1. 数字运算和循环结果图

## 1.2 偶数平方和

### 1.2.1 实验过程（包括是否遇到问题及如何解决）

解答：此题与前面的类似，不过这里求的是偶数的平方和，编写好判断偶数的函数 `isEven` 和平方函数 `square` 之后，JavaScript 提供了自身的语法，通过 `filter` 方法调用判断函数过滤掉非偶数，再通过 `map` 方法逐元素调用平方函数，最后通过 `reduce` 方法传入自定义的加和函数，并规定从 0 开始加和，最后返回 1-10 之间的偶数平方和；这些函数是需要额外查询才能知道的。

### 1.2.2 实验结果

解答：运行补充完的脚本，可以得到如图 2 所示的结果，可以看到数组中仅填充了 1-10 之间的偶数平方，最后输出了加和为 220，与预期一致，证明代码编写成功。



图 2. 偶数平方和结果图

## 1.3 动态计数器

### 1.3.1 实验过程（包括是否遇到问题及如何解决）

解答：实验首先需要定义一个全局计数器用于更改后的更新，接着使用文档流 DOM 对“加一”和“减一”两个按钮添加事件处理函数，选择类型为“click”规定为点击时触发；而题目规定计数必须在 0-10 之间，所以每次操作时需要判断一下操作是否有效，当不在合法范围时则触发相应的警告信息，计数器不变，通过 DOM 的 `innerText` 添加到 HTML 上，否则计数进行相应操作，最后都需要将 `counter` 显示在网页上。

### 1.3.2 实验结果

解答：按照上述思想补充完脚本后，运行结果部分截图如图 3 所示：

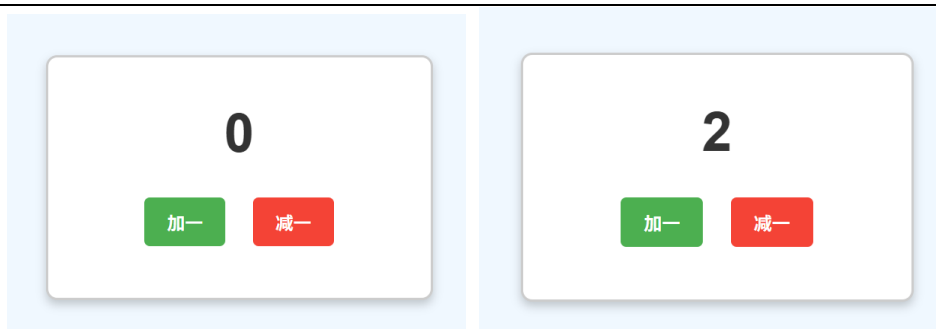


图 3. 动态计数器初始图（左）点击后效果（右）  
当计数不在合理范围内时，计数会不变，同时出现警告信息如图 4 所示：

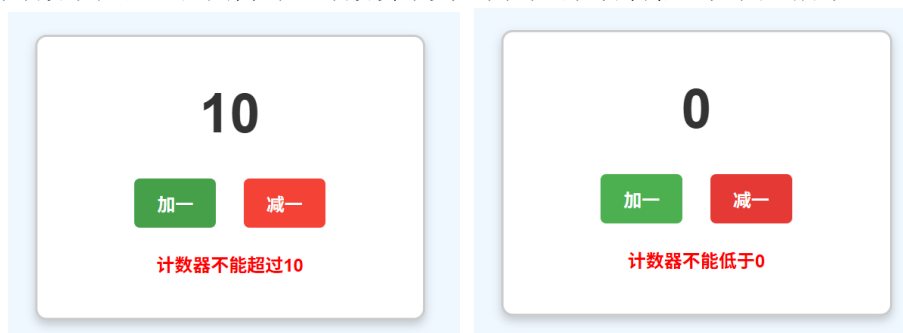


图 4. 计数超过上限（左）计数低于下限（右）

## 1.4 商品清单生成器

### 1.4.1 实验过程（包括是否遇到问题及如何解决）

解答：按照实验流程，首先需要手动定义商品清单数组 `products`，每个元素是一个字典，接着遍历每个字典内容，由于题目要求对每个商品使用 `li` 元素，因此这里使用 DOM 的 `createElement` 方法对每个字典创建 `li` 元素，并设置每个元素的内容为题目的指定字符串格式 `'${product.name} - $$${product.price}'`，接着对每个 `li` 元素根据价格不同设置不同的颜色，最后，也是容易忽略地，**需要将每个 `li` 元素添加到 `ul` 中。**

### 1.4.2 实验结果

解答：根据上述思路编写脚本后，运行得到如图 5 所示的结果：



图 5. 商品清单运行结果

## 1.5 学习主页优化

### 1.5.1 实验过程（包括是否遇到问题及如何解决）

解答：此实验主要是 `svg` 矢量图形的使用，`svg` 也像 `HTML` 的标签一样，不过需要去网上复制其相应的样式，在对应的地方写入即可；同时根据要求，为每个图标添

加相应的样式，导航栏悬浮时变色的效果，以及为背景图片（background-image）设置背景渐变和叠加，具体通过设置类选择的 nth-child 子元素即可，不过值得注意的是，这里动画效果是在导入的外部样式中。

### 1.5.2 实验结果

解答：按照上述思路补充完代码，运行脚本如图 6 和图 7 所示：

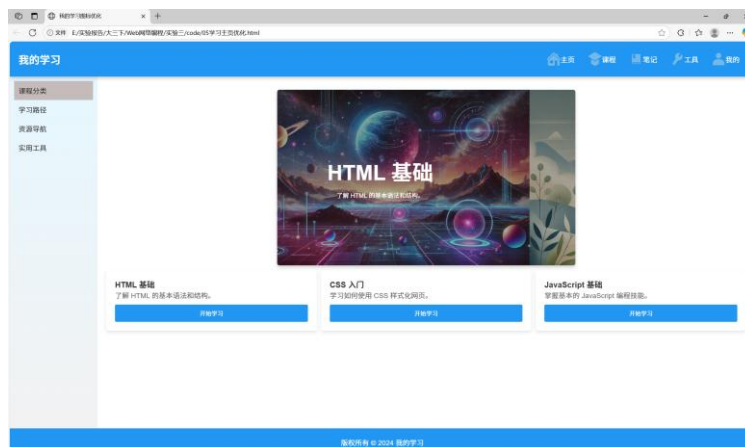


图 6. 学习主页初始页面

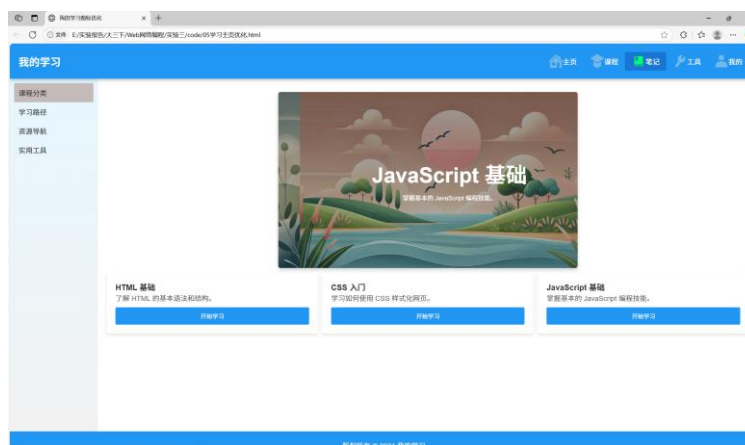


图 7. 学习主页点击效果

## 1.6 星级评分

### 1.6.1 实验过程（包括是否遇到问题及如何解决）

解答：根据所给的框架和注释，总体可以分为三部分，一是高亮鼠标选中的星星，二是鼠标点击后固定当前的星星并计算当前的评分，三是重置按钮点击后重置星星和评分；这三部分根据给出的注释一步一步完成大体都可以完成，但在此之前还需要初始化星星对象 stars，并为每个星星添加鼠标事件（这里是“mousemove”、“moveleave”和“click”），使得鼠标在星星上移动时能够实时高亮，移开时能够恢复，点击时能够计算新的评分并且显示；此外，还需要为重置按钮设置对应的“click”逻辑；总体而言，首先完成函数编写，初始化时为每个对象添加事件并进行对应逻辑的编写（调用声明函数，更新变量等）。

### 1.6.2 实验结果

解答：根据上述思路补充代码，运行脚本后可以得到如图 8 所示的效果：



图 8. 初始评分界面（左） 选定星星后效果（右）

## 1.7 待办事项

### 1.7.1 实验过程（包括是否遇到问题及如何解决）

解答：此实验主要是考察怎么在 HTML 界面上添加、删除和修改元素；整体首先获取 HTML 上的元素，CSS 样式已经定义好了，我们只需要根据需求添加这些元素即可，验证输入框的内容合法后，创建文本框、文本和删除按钮，**注意元素之间的父子关系**，最后还需要为删除按钮添加事件监听器，点击后从 `ul` 中删除对应 `li` 元素。

### 1.7.2 实验结果

解答：按照上述思路补充代码，运行脚本可以得到图 9 效果，添加一些待办事务后，可以得到图 10 的效果：

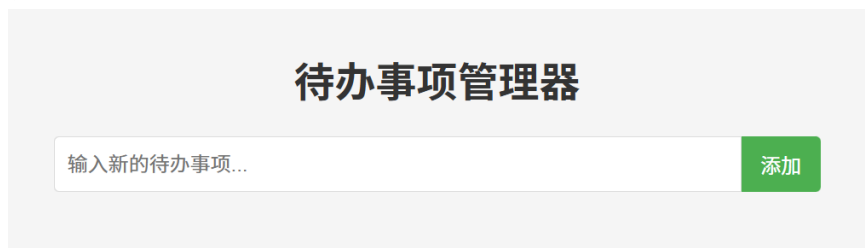


图 9. 待办事务管理器初始界面

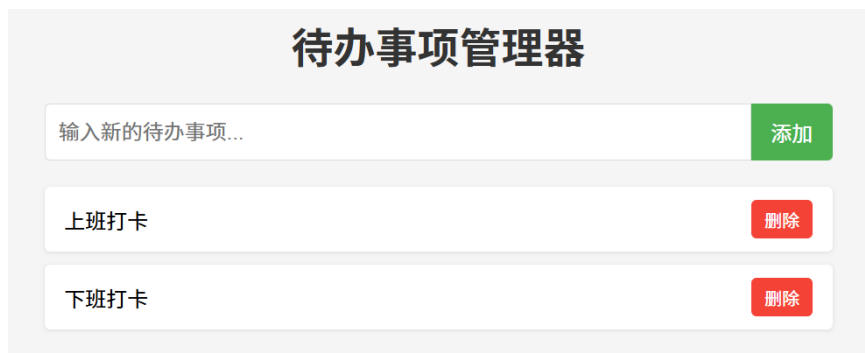


图 10. 待办事务管理器添加事务后的效果

## 1.8 学习主页再次优化

### 1.8.1 实验过程（包括是否遇到问题及如何解决）

解答：前面的实验已经完成了学习主页背景图、图标的优化，本次再次优化打算从侧边栏优化入手，从 <https://www.iconfont.cn/> 阿里巴巴矢量网下载对应的四个图标，复制对应的 `svg` 到相应的代码区域（**注意要删除 `svg` 修饰中的 `fill` 字段，方便整体颜色的管理**），设计整体的样式和之前导航栏的图标类似，但由于图标和文字并没有水

平对齐，所以需要为这几个 svg 单独处理使得图标和文字居中对齐，代码如下：

```
.side-menu-item svg {  
    vertical-align:middle; /*图标和文字居中对齐*/  
    margin-right:8px;  
}
```

### 1.8.2 实验结果

解答：按照上述要求完善代码，运行脚本得到如图 11 的效果：

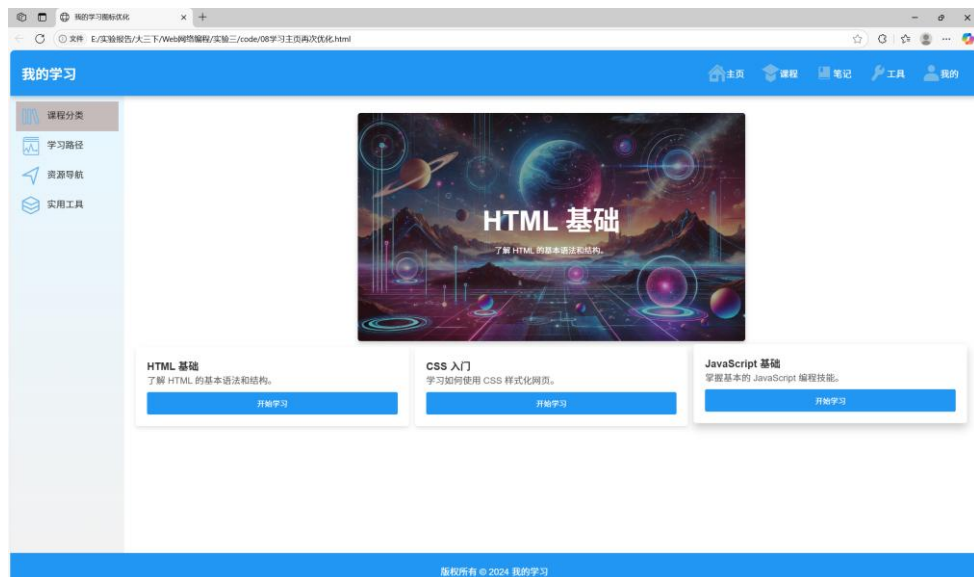


图 11. 学习主页再次优化的效果

## 1.9 英雄快跑游戏

### 1.9.1 实验过程（包括是否遇到问题及如何解决）

解答：本次实验应该是整体下来难度较大的实验了，要求我们对实验代码整体要了解才能作答；除了按照所给注释进行代码编写之外，有两个地方需要额外注意：

- 一个是角色图片切换和帧计数器，实验给定了四张角色图片，实验需要设定什么时候角色图片的更新，这里采用每 10 帧更新一次角色图片的方式，对 4 张角色图片进行轮流更新；
- 另一个是与坑洞进行碰撞的逻辑，由于坐标原点是在左上角，所以越往下 y 值越大，而且实验设置大于 250 的部分作为地面，所以小于 250 的部分是浮空状态，因此判断角色与任何一个坑洞的碰撞代码应该如下：

```
return this.pits.some((pit) => {  
    // x 位于坑洞之间，y 低于地面  
    if(this.player.x >= pit.x &&  
        this.player.x + this.player.width <= pit.x + pit.width &&  
        this.player.y >= 250)  
        return true;  
    else  
        return false;  
});
```

### 1.9.2 实验结果

解答：补充完实验所给代码后，特别是前面的两个细节后，运行得到的图 12 的效果：



图 12. 英雄快跑效果

## 1.10 拖拽排序

### 1.10.1 实验过程（包括是否遇到问题及如何解决）

解答：实验要求对数组中的素数进行降序排序，并计算平均值保存两位小数，这与前面的实验有些许类似，也是用了 `filter` 函数调用了 `isPrime` 进行素数的过滤，得到的素数数组后，调用 `sort` 函数（自定义降序逻辑）对该数组进行排序，最后编写函数计算平均值即可，不过这里值得注意的是保存两位小数函数是 `toFixed(2)`。

### 1.10.2 实验结果

解答：按照前面的思路编写代码后，在控制台打印出目标值，结果如下所示：

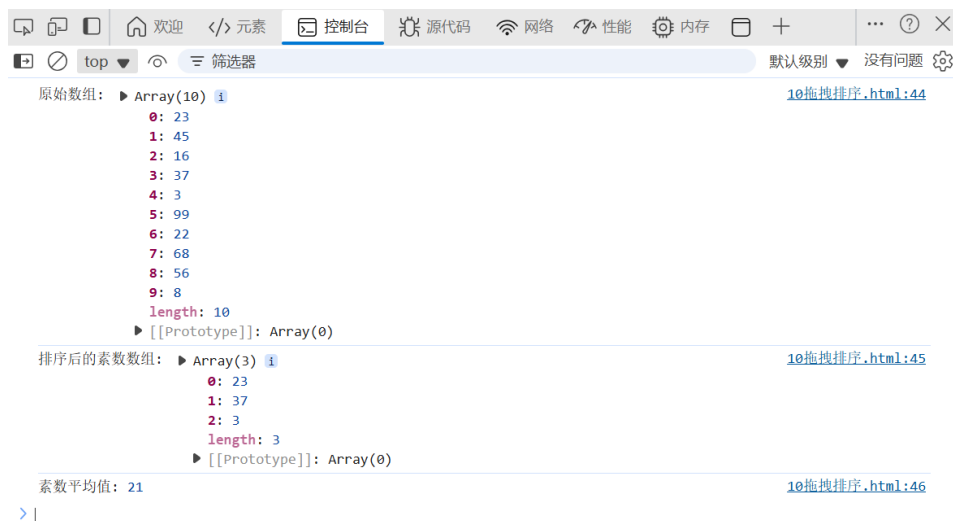


图 13. 拖拽排序效果

## 1.11 通讯录管理

### 1.11.1 实验过程（包括是否遇到问题及如何解决）

解答：本次实验使用了 `IndexedDB`，这是一种在浏览器中存储大量结构化数据的 `Web API`，它允许开发者在客户端存储和检索数据，支持事务操作和索引查询，是 `Web` 应用实现离线功能的重要技术；通过这种技术，我们不需要在本地创建相应的数据库。



不过值得注意的是，IndexedDB 采用的是异步 API，从而避免阻塞主线程。

实际使用时需要注意以下用法：

- (1) 打开数据库：（数据库名称，版本号）

```
const request = indexedDB.open("contactsDB", 1);
```

- (2) 创建/更新数据库结构：

```
request.onupgradeneeded = function(event) { .....};
```

- (3) 操作数据：（开启事务，获得对象存储）

```
var transaction = db.transaction(["contacts"], "readonly");  
var objectStore = transaction.objectStore("contacts");  
var request = objectStore.getAll();  
// 异步操作的回调函数，分别用于处理请求成功和失败的情况  
request.onsuccess = function(event) {};  
request.onerror = function(event) {};
```

### 1.11.2 实验结果

解答：按照上述用法补充相应的 IndexedDB 代码，运行代码效果如图 14 所示：

## 手机通讯录管理器

添加

通讯录为空，请添加联系人。

图 14. 通讯录初始运行界面

为该通讯录添加一些联系人后，效果如图 15 所示，如果有重复则系统会弹出是否覆盖的信息；

此页面显示  
该电话号码已存在，是否覆盖？

确定 取消

王五

添加

张三 - 123456789 删除

李四 - 234567891 删除

王五 - 345678912 删除

图 15. 通讯录添加联系人效果

## 1.12 WebWorker

### 1.12.1 实验过程（包括是否遇到问题及如何解决）

解答：本次实验使用到了 WebWorker 技术，它运行浏览器主线程之外创建独立的后台线程，用于执行耗费资源或者时间的任务，从而避免阻塞用户界面的相应。

实际使用时要注意：

- (1) 主线程创建 Worker：通过 `new Worker("...js")` 启动一个后台线程；
- (2) 双向通信：通过 `postMessage` 发送信息，通过 `onmessage` 接收响应；
- (3) 任务处理：Worker 线程独立执行脚本，不访问 DOM 但可以操作数据

实验中的 WebWorker 的 js 文件要求每 1000 次计算发送一次进度，最后返回总的素数个数，代码编写如下：

```
self.onmessage = function(event) {  
    let count = 0;  
    const {start, end} = event.data;  
    for (let num = start; num <= end; num++) {  
        //每 1000 个数发送一次进度  
        if (num % 1000 === 0) {  
            self.postMessage({type: 'progress', current: count, total: num});  
        }  
        if (isPrime(num)) {  
            count++;  
        }  
    }  
    self.postMessage({type: 'result', count: count});  
}
```

### 1.12.2 实验结果

解答：按照上述要求编写 webworker 的代码后，修改圆圈动画重置代码，优化素数的计算函数之后；由于 Webworker 不能直接在 HTM 执行（浏览器的安全限制，协议之间不相互支持），所以使用 Node.js 搭建一个简单的服务器，服务器环境两者同源可以被正常加载，运行效果如图 16 所示：



图 16. Webworker 运行效果

### 1.13 用户注册

#### 1.13.1 实验过程（包括是否遇到问题及如何解决）

解答：本次实验使用了 AJAX 异步通信技术，核心特点是不在重新加载整个网页的情况下，与服务器进行数据的交换并更新部分网页内容；实验中主要使用这个技术用于两个场景：

- a. 用户名注册：当用户点击用户名输入框之外的地方时异步检查用户名是否存在；
  - b. 注册提交：表单验证通过后，将注册信息提交到服务器
- 整体格式如下：

```
$.ajax({  
    url: "http://172.31.233.204:5000/user/register", // 请求地址  
    type: "POST", // 请求方法  
    contentType: 'application/json', // 发送数据的格式  
    data: JSON.stringify({...}), // 转换为 JSON 格式的请求数据  
    success: function(response) { ... }, // 成功响应处理  
    error: function() { ... } // 错误响应处理  
});
```

此外，实验还要求使用 jQuery 库，该库中的核心标识符是“\$”，可以通过它来快速选择元素，同时也为 AJAX 提供了更加简洁的接口。

### 1.13.2 实验结果

解答：按照上述思路补充完代码，运行脚本效果如图 17 所示：



图 17. 初始界面（左）注册成功页面（右）

## 2. 设计并实现某主题网站的账户模块，包括

### 2.1 构思、设想

账户模块是非常常见的一个模块，并且也是一个持续在改进优化用户体验，同时持续提高用户账号安全的模块。参考前述用户注册实验，并结合调研的网站，请精心设计某主题网站的账户模块相关页面，从用户体验和安全保障 2 个方面，对该模块的注册、登录等功能进行设计。

解答：本次实验本人打算接着之前实验的内容，做一个名为 SoundWave 的音乐流媒体网站账户功能，包括登录和注册功能，整体的网站形式更加现代化，同时通过视觉、功能实现、多方式登录以及明显的错误提醒给用户带来多元化的体验，并通过密码强度评估、密码确认和数据加密等形式保证安全性。

### 2.2 设计实现及安全优化

针对前述构思设想情况，你是如何设计并实现该页面的。请结合“代码片段”选取用户体验和安全保障 2 个要点进行介绍。

解答：由于本实验为了界面更加美观，使用了 TailWind CSS，CSS 样式代码较多，这里不全部加以展示，详细代码请见提交附件。

### (1) 用户体验优化

**直观的表单设计：**表单布局清晰，标签明确，使用图标增强视觉引导，每个输入字段都有明确的占位符文本；实际通过

**实时表单验证：**包括用户名长度检查、邮箱格式验证、密码强度实时评估（显示密码强度指示器和具体要求）和密码一致性检查；

**视觉反馈：**输入验证结果的即时视觉反馈（颜色变化、图标提示）、操作成功的 toast 通知和平滑的动画过渡效果（模态框打开与关闭、表单切换）等；

**多方式登录：**除了传统的邮箱与密码登录，还提供了 Google、Facebook 和 Apple 的第三方登录选项；（当然这里只显示了图标，详细链接并没有去做）

**密码可见性切换：**允许用户临时显示密码，方便确认输入，具体通过添加 JS 监听器，对 password 的属性在 text 和 word 之间进行切换，同时通过切换 i 标签达到“显式”和“隐藏”的状态（也就是眼睛睁开和闭上的效果）；下面是一个代码 demo：

```
oggleLoginPassword.addEventListener('click', () => {  
    const type = loginPassword.getAttribute('type') === 'password' ? 'text' : 'password';  
    oginPassword.setAttribute('type', type);  
    toggleLoginPassword.querySelector('i').classList.toggle('fa-eye');  
    toggleLoginPassword.querySelector('i').classList.toggle('fa-eye-slash');  
});
```

**清晰的错误提示：**当输入不符合要求时，提供具体的错误信息和修复建议；以用户名输入为例，网站设计的是满足长度至少为 4，如果低于则显示“×用户名”表示用户名不合格，合理则显示“✓用户名”；下面是个代码 demo：

```
registerName.addEventListener('input', () => {  
    if (registerName.value.length < 4) {  
        usernameFeedback.classList.remove('hidden');  
        registerName.classList.add('border-red-500');  
        registerName.classList.remove('border-gray-300');  
    } else {  
        usernameFeedback.classList.add('hidden');  
        registerName.classList.remove('border-red-500');  
        registerName.classList.add('border-gray-300');  
    }  
});
```

### (2) 安全保障优化

**密码强度评估：**实时检查密码强度，要求至少包含 8 个字符、大小写字母和数字，帮助用户创建更安全的密码；

**数据加密：**表单使用 HTTPS（通过安全的服务器）确保数据传输过程中的安全性；

**密码确认：**注册时要求用户再次输入密码，防止输入错误；

**账户保护提示：**清晰的服务条款和隐私政策链接，保护用户权益。

## 2.3 效果呈现

解答：运行编写的脚本，界面效果如图 18 所示：（界面借鉴了主流网站的流媒体形式）

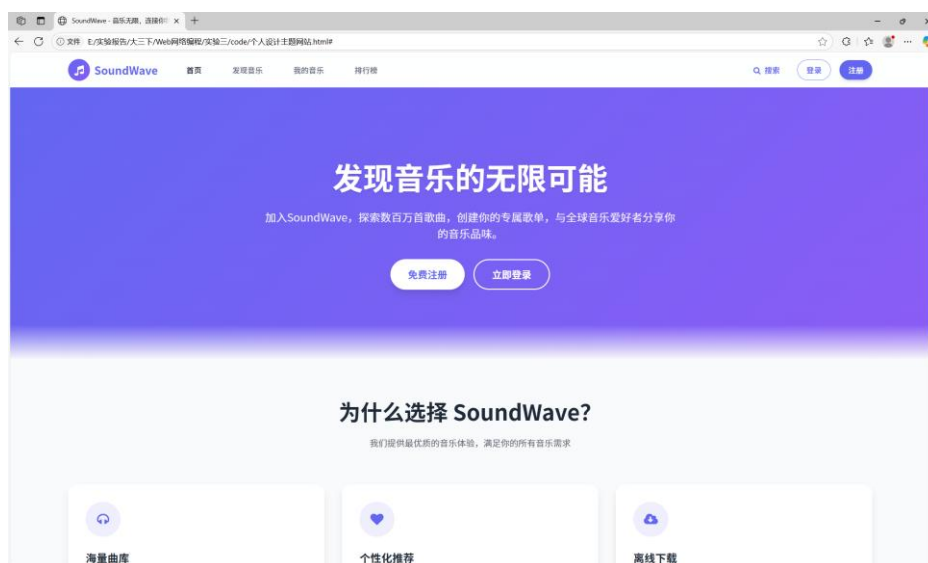


图 18. 个人设计网站效果

点击右上角的“登录”填写邮箱和密码进行登陆，效果如图 19 所示，其中右图进行了错误测试，当邮箱格式不对时会弹出错误。



图 19. 初始登陆页面（左）填写信息效果（右）

点击右上角的“注册”或者通过“登录”页面下面的“立即注册”切换到“注册”页面，需要用户填写用户名、电子邮箱、密码以及重复密码的安全性验证，同时需要用户打勾相应的“服务条款”和“隐私政策”，符合目前常见的注册页面设计；同时对密码还进行了校验，至少 8 个字符、包含大小写且包含数字，不同的密码组合根据强弱分成了四种强度，不过代码并没有强制一定要达到某种强度，至少符合一种即可。效果如图 20 所示：

登录

注册

×

创建账户

加入SoundWave, 开启你的音乐之旅

用户名

你的用户名

电子邮箱

your@email.com

密码

至少8个字符, 包含字母和数字

至少8个字符

包含大写字母

包含小写字母

包含数字

确认密码

再次输入密码

☐ 我同意 服务条款 和 隐私政策

注册

已有账户? 立即登录

登录

注册

×

创建账户

加入SoundWave, 开启你的音乐之旅

用户名

breaker

电子邮箱

1234@qq.com

密码

非常强

至少8个字符

包含大写字母

包含小写字母

包含数字

确认密码

☒ 我同意 服务条款 和 隐私政策

注册

已有账户? 立即登录

图 20. 初始注册页面（左）填写信息效果（右）

## 2.4 思考小结

与常见各种“用户注册、登录”对比，你自己如何评价上述功能的设计、实现。

解答：本次实验设计的登录注册页面大体实现了基础的功能，整体设计符合主流网站的样式，界面美观，用户体验好，安全性强，符合现代 Web 应用的设计标准；不过由于时间问题，整体网站的设计并不是很完善，包括服务条款的必须选定，注册或者登陆成功后显示个人用户，用户名不能重复等等细节，还是值得继续改进的。

## 实验结论：

1. 完成实验后，当你看到自己刚做完的账号页面，你有何感想？

解答：其实自己编写一个账号页面并不难，在有 HTML 和 CSS 的基础上加入了 Javascript，能帮助将一个账号页面基础功能基本完成了，但是整体美观的细节还是挺难把握的，本次实验还是采用的 TailWind CSS，方便对每个元素进行操作才能得到这么好的效果。整体做下来，难的是设计，累的是样式的调整，而且这个账号页面还是整个页面的一部分，剩下好多功能还没有实现，侧面说明了前端确实很累。

## 心得体会：

主要说明你对 JS 的理解和整体感受。

解答：JS 是前端灵活的交互工具，让静态界面有了响应式灵魂，虽语法简洁但能精准控制行为，是前端构建交互体验的核心支点，用起来很有“四两拨千斤”的巧妙感，把逻辑转化为直观交互，让网页不再呆板，是前端开发者和页面交互沟通的“魔法棒”，用代码指令赋予元素交互生命力，不断拓展网页应用的可能性，虽调试时偶尔头疼，但实现功能后的成就感拉满。

指导教师批阅意见：

成绩评定：



指导教师签字：Basker George  
2025 年 06 月 28 日